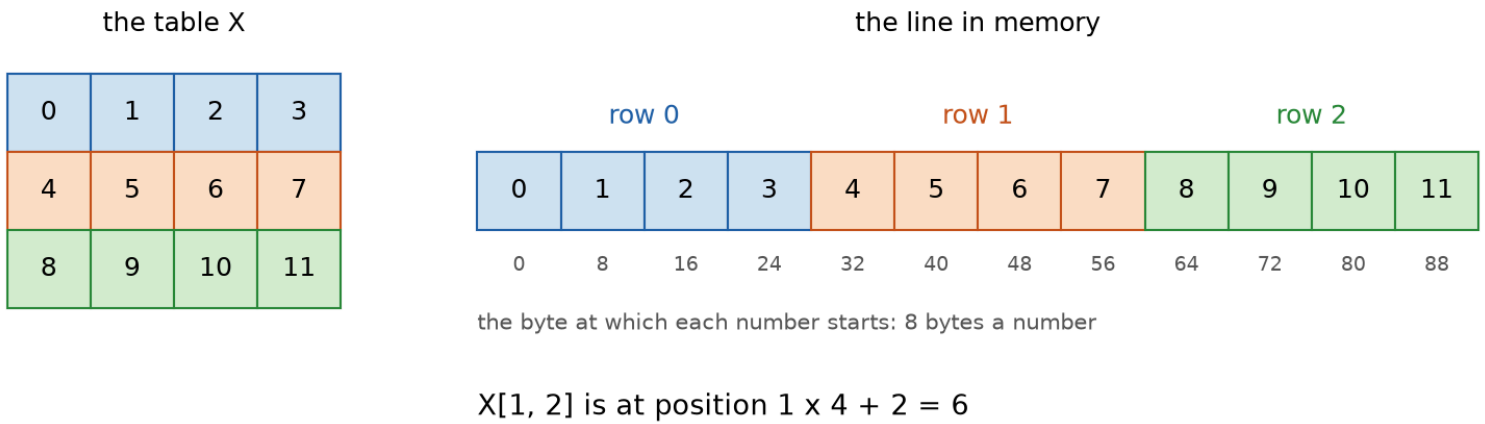
CS 301 · MEETING 12 · RECAP 2

What Big O Leaves Out

How storage layout, access order, and cache conflicts affect running time.

A matrix is one line of numbers




```
the line in memory: [ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11.]
bytes for one number: 8   strides: (32, 8)
```

- **Row order**: the rows one after another
- X[i, j] is at position i × m + j

The transpose: the same line, read another way

```
X.T has shape (4, 3) and strides (8, 32)   same memory as X: True
in column order, the line is: [ 0.  4.  8.  1.  5.  9.  2.  6. 10.  3.  7. 11.]   strides: (8, 24)
```

- `X.T`: the two strides swapped; no number moves
- **Column order**: the columns one after another (`np.asfortranarray`)
- Shape and strides describe the view; the data stays in place

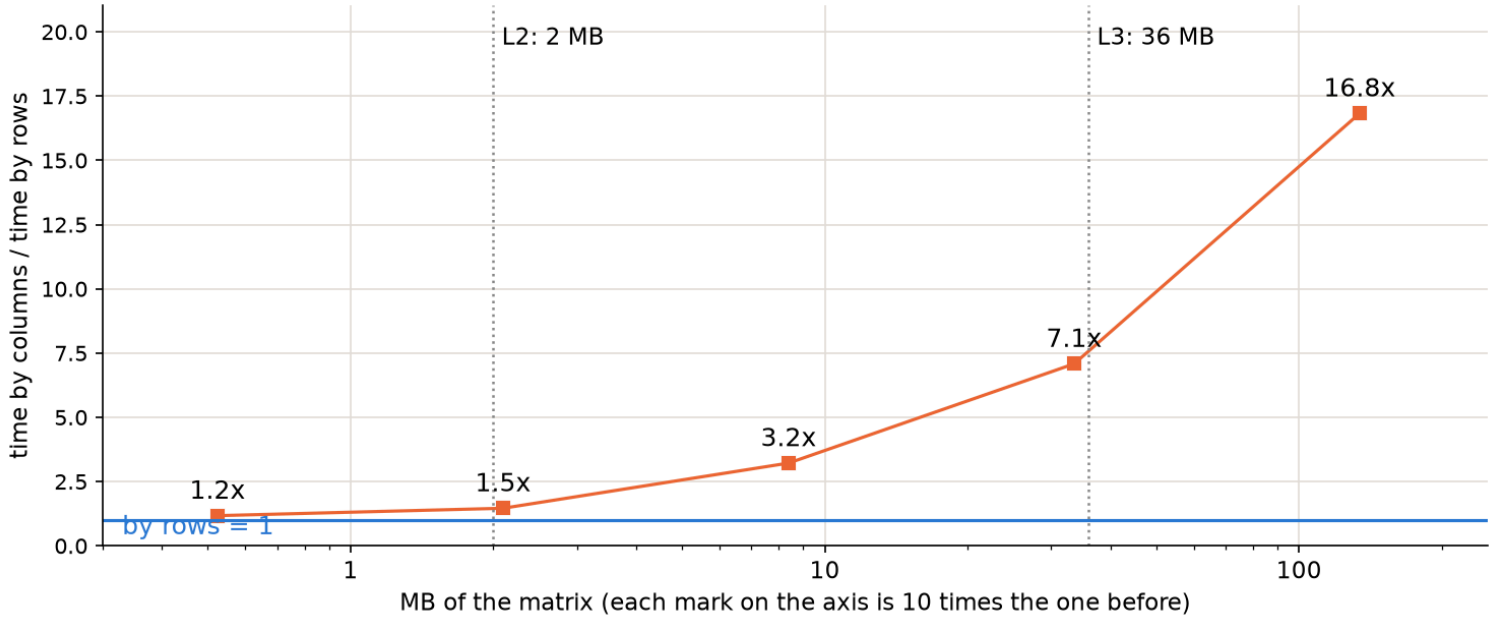
By rows and by columns, in Python

```python
def sum_by_columns(X):
    n, m = X.shape
    total = 0.0
    for j in range(m):
        for i in range(n):
            total += X[i, j]
    return total
```

```
2000 x 2000 numbers, 32 MB
by rows:    0.361 s, 90 nanoseconds for each number
by columns: 0.368 s, 92 nanoseconds for each number
by columns / by rows: 1.02
```

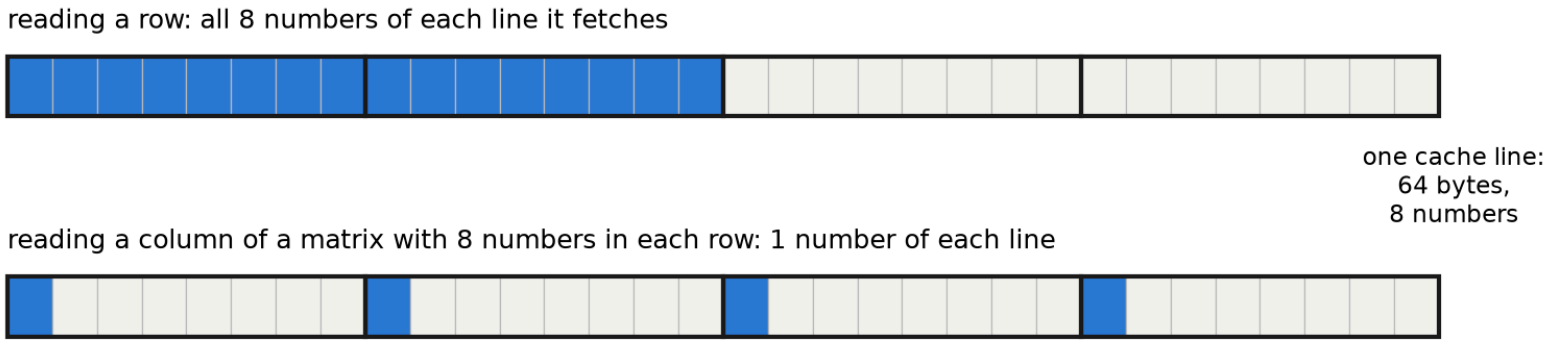
- Compare the times: interpreter overhead can mask access costs
- The interpreter's work for each number hides the memory

By rows and by columns, with NumPy



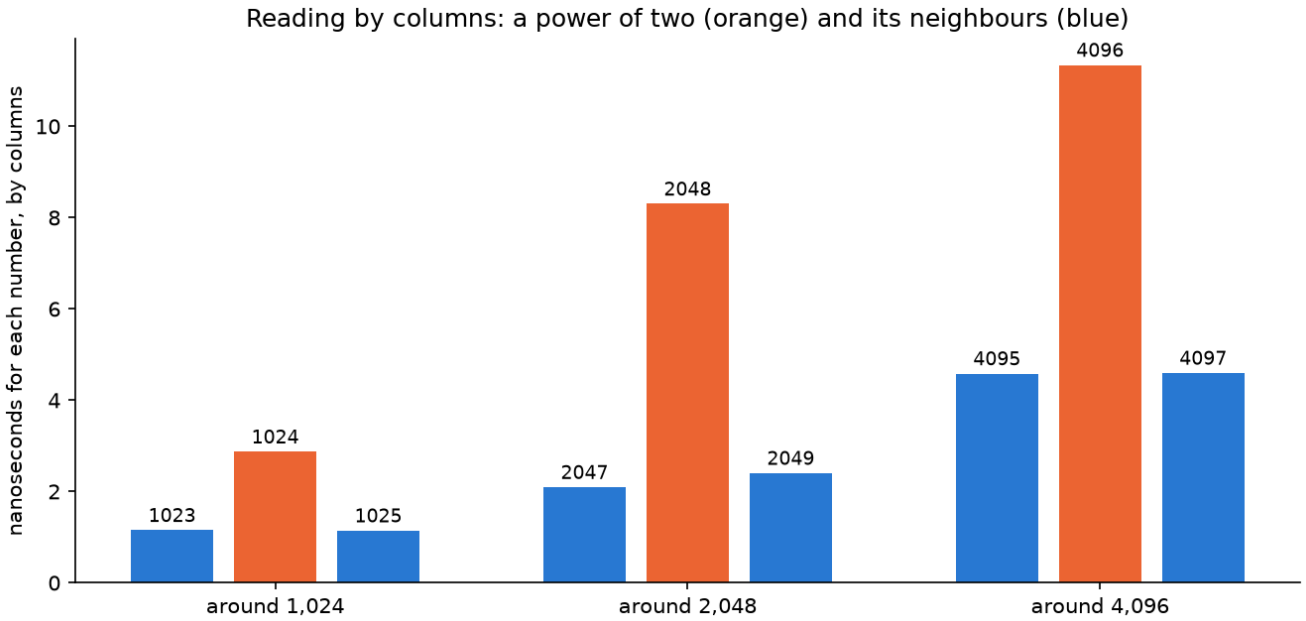
- A loop over rows or columns; NumPy adds up each one
- Column reads become more expensive as cache reuse falls

A cache line: 64 bytes



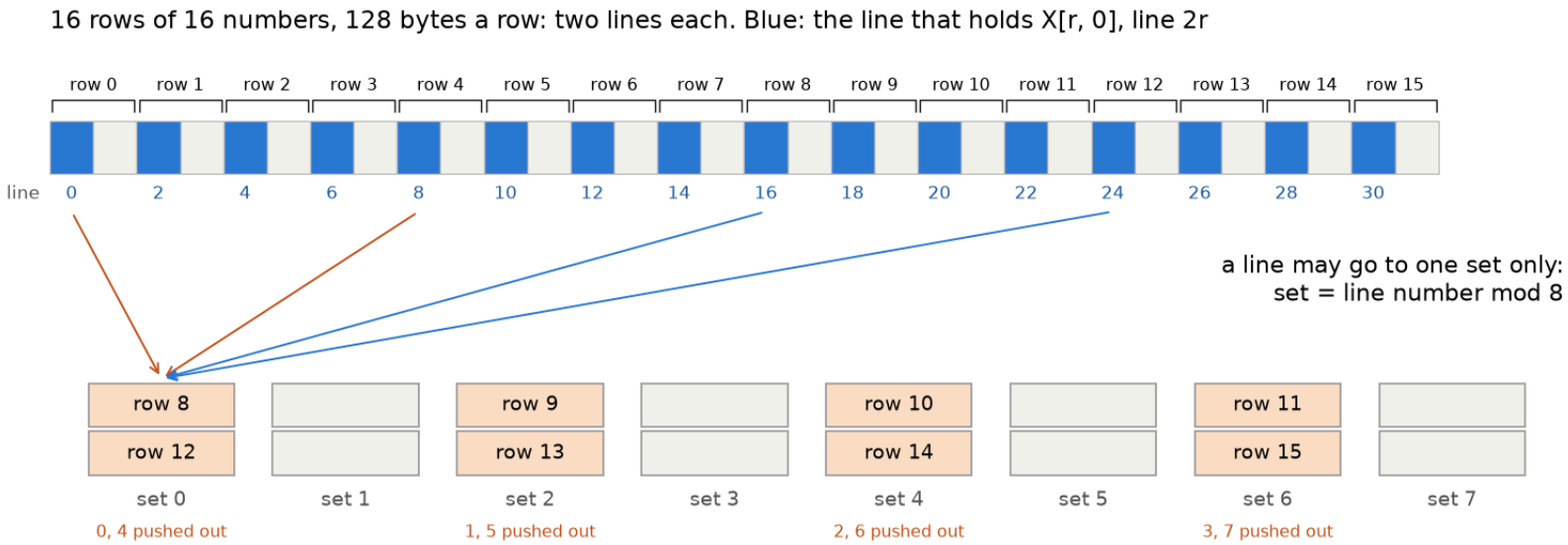
- The memory hands the cache 64 bytes at a time: 8 numbers
- A row uses all 8; a column uses 1, then jumps a row ahead
- Read in order, the next lines are fetched before they are asked for

A power of two, and one more



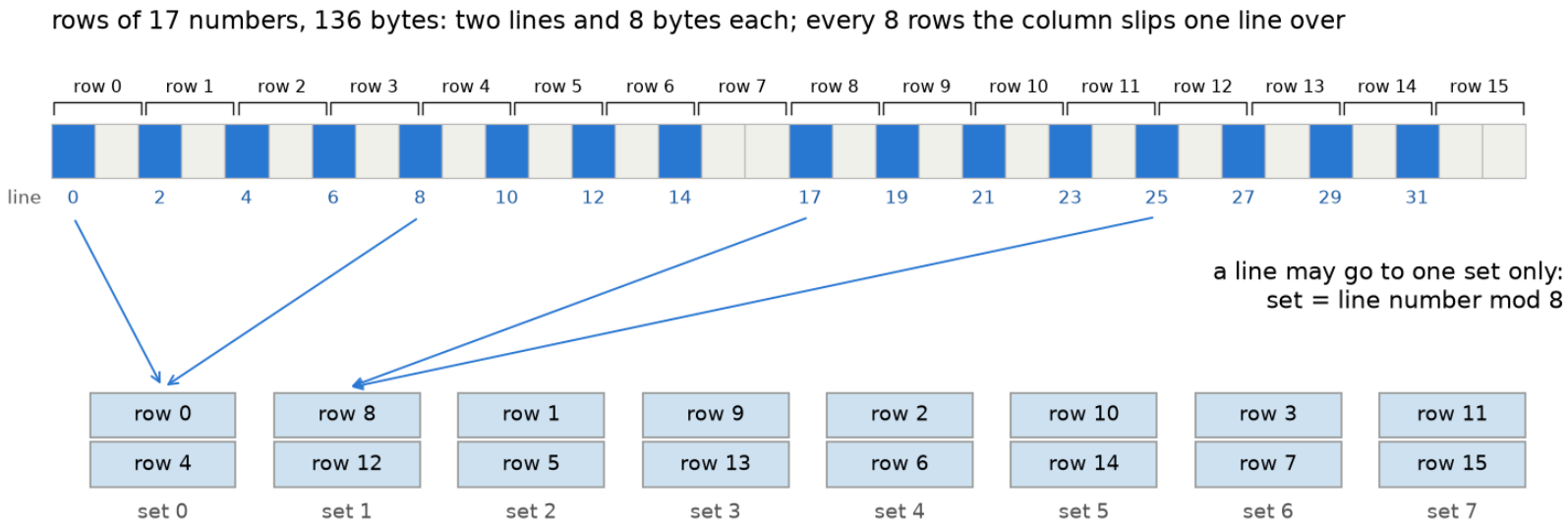
- Here, column reads are slower at the power-of-two sizes
- Nearby sizes reduce the penalty in this measurement

Sets: where a cache line may go



- The cache keeps **lines**: 8 side-by-side numbers, 64 bytes
- This cache model: set = line number mod number of sets
- Here: 16 first-column lines compete for 8 slots in 4 sets

One number more in each row, and the lines spread



After column 0: all 16 fetched lines fit, two per set

- Rows of 17 numbers: every 8 rows, the column slips to the next line and the next set
- L2 model, aligned column 0, 2,048 rows: 2,048 per row reaches 8 sets; 2,049 reaches all 2,048

Padding: one more number in each row

```
The columns of a matrix of 2,048 by 2,048 numbers:
stored as it is, rows of 2,048 numbers:     25.6 milliseconds
padded, rows of 2,049 numbers:             10.9 milliseconds
as it is / padded: 2.4
```

- Rows of 2,049 numbers, the last one unused: the same table
- `np.zeros((n, m + 1))[:, :m]`: change the stride, keep the shape

Column sums, read by rows

```
The sums of the 4,096 columns of a matrix of 134 MB:
a loop over the columns, X[:, j].sum():      189.2 milliseconds
column_sums_by_rows, a loop over the rows:     6.0 milliseconds
NumPy's own X.sum(axis=0):                     4.6 milliseconds
```

- The same m sums: running sums, one row added at a time
- Reading in the order of the line; NumPy's `X.sum(axis=0)` does this itself

What NumPy does by itself

operation	ms	the same numbers, another order	ms	ratio
X.sum(axis=1)	7.4	X.sum(axis=0)	4.5	0.6
X @ v	2.8	F @ v, F in column order	3.9	1.4
X.copy()	22.5	X.T.copy()	204.8	9.1
X + X	24.2	X + X.T	206.9	8.6

- NumPy can accumulate column sums while reading contiguous rows
- `X + X.T`, `X.T.copy()`: one of the two must be read by columns

A transposed operand in a product: a flag, not a copy

```
product, n = 4,096        PyTorch     NumPy   what the library is handed
X @ W                     0.194 s   0.227 s   both stored by rows
X @ W.T                   0.196 s   0.237 s   W.T by columns: a flag, no copy
X.T @ W                   0.194 s   0.230 s   X.T by columns: a flag
X.T @ W.T                 0.196 s   0.229 s   both flagged
X[:, ::2] @ W             0.201 s             no unit stride: copied first
the copy by itself        0.006 s
```

- `X @ W.T`: the library is told op(W) = transpose; it repacks blocks in the cache as it uses them
- Transpose flags avoid a full copy; compare the four measured times
- No unit stride, `X[:, ::2]`: copied first, one extra pass and a new buffer, not a new $O(n^3)$

The same product on this laptop's cores

- **architecture-dependent**: measured on the laptop that built these slides, an Intel Core i9-14900HX with 8 performance cores and 16 efficiency cores

```
X @ W, n = 4,096, PyTorch, pinned to      median    min    max   6 rounds, seconds
the 8 performance cores, 8 threads          0.26   0.26   0.28
the 16 efficiency cores, 16 threads         0.23   0.23   0.24
all 24 cores, 24 threads: the default       0.20   0.20   0.22
```

- The same count, the same code, the same library: the cores it runs on decide the seconds
- Core placement, thread scheduling, and a shared power budget affect throughput
- A timing is a sample of a machine state: compare variants side by side, in rounds, by the median

The same O( ), different seconds

what O( ) leaves out	measured before the meeting	how much
the interpreter	Python's loops against NumPy	about 100 times
the order of reading	a matrix by rows or by columns	1 to about 20 times
the cache's sets	2,048 columns against 2,049	2 to 4 times
where the numbers are	`a + b` in the cache against main memory	about 5 times
operations per byte, and the cores	`a + b` against the product, one CPU	hundreds of times
the processor	the T4 against Colab's CPU: your posts	22 to 40 times

- Use counts and measured throughput together to estimate time